

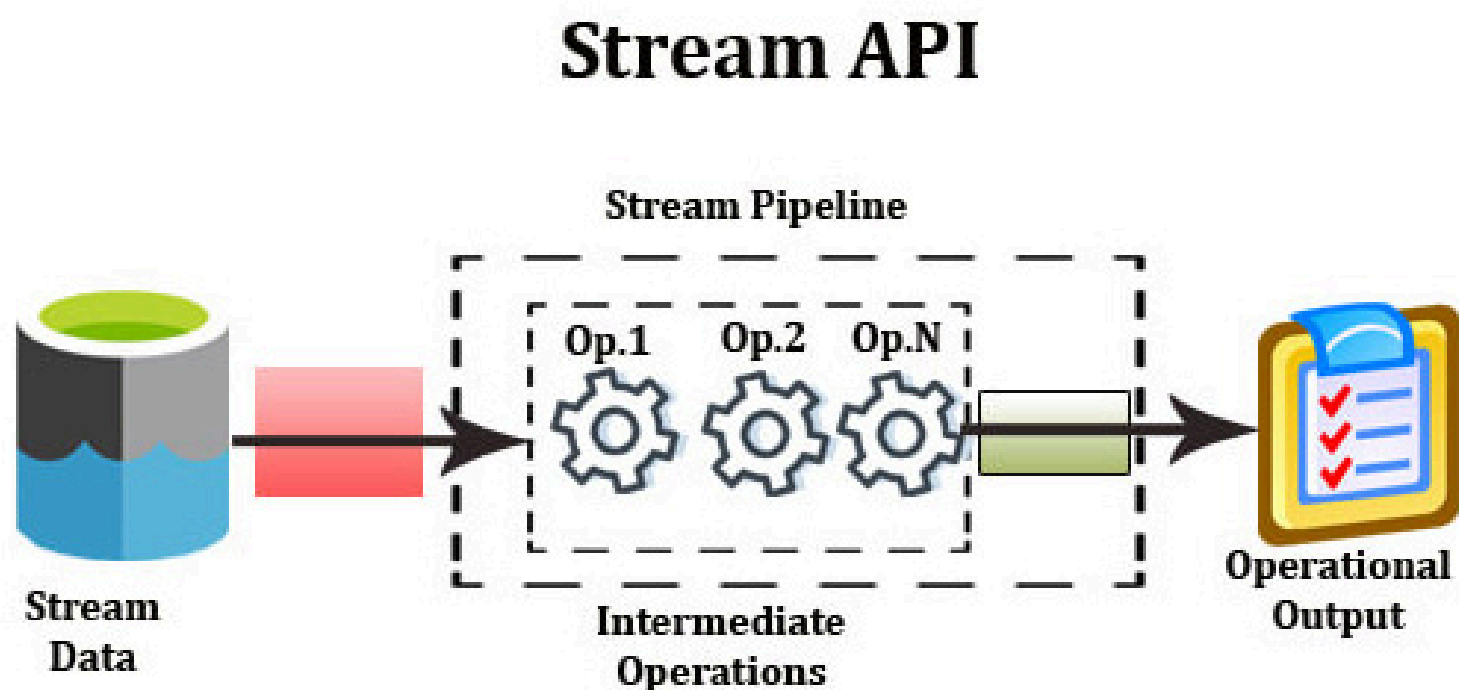
JAVA
CHEATSHEET
STREAM API
PART - 1



@java_in.depth

WHAT IS THE JAVA STREAM API?

The Stream API in Java provides a modern way to manipulate and process data collections, such as arrays, lists, or sets, in a functional style. Streams are not data structures but rather pipelines for processing data, offering features like filtering, mapping, reducing, and more.



1. CREATING STREAMS

Streams process collections in a functional style in Java.

```
List<Integer> numbers = List.of(1,2,3,4,5);

// Stream from collection
numbers.stream();

// Stream from array
Arrays.stream(new int[]{1,2,3});

// Stream using Stream.of
Stream.of("A", "B", "C");
```

Streams allow operations like filtering, mapping, and collecting results efficiently.

2. FILTER()

Used to remove elements that don't match a condition.

```
List<Integer> numbers = List.of(1,2,3,4,5,6);

List<Integer> even =
    numbers.stream()
        .filter(n -> n % 2 == 0)
        .toList();

System.out.println(even);
```

`filter()` keeps only elements that satisfy the predicate.

3. MAP()

Transforms each element into another value.

```
List<String> names =  
List.of("java", "spring", "backend");  
  
List<String> upper =  
names.stream()  
    .map(String::toUpperCase)  
    .toList();  
  
System.out.println(upper);
```

map() is used when you want to modify each element of a stream.

4. SORTED()

Sorts elements in natural or custom order.

```
List<Integer> numbers = List.of(5,2,9,1);

List<Integer> sorted =
    numbers.stream()
        .sorted()
        .toList();

System.out.println(sorted);
```

Custom sorting example:

```
.sorted((a,b) -> b - a)
```

5. LIMIT() & SKIP()

Used for slicing streams.

```
List<Integer> numbers = List.of(1,2,3,4,5,6,7);  
  
numbers.stream()  
    .limit(3)  
    .forEach(System.out::println);
```

Skip example:

```
numbers.stream()  
    .skip(3)  
    .forEach(System.out::println);
```


6. COLLECT()

Used to convert streams into collections.

```
List<String> names =  
List.of("Java", "Spring", "API");  
  
List<String> list =  
names.stream()  
    .collect(Collectors.toList());
```

collect() is one of the most used terminal operations.

7. REDUCE()

Used for aggregation operations.

```
List<Integer> numbers = List.of(1,2,3,4);

int sum =
    numbers.stream()
        .reduce(0, (a,b) -> a + b);

System.out.println(sum);
```

Commonly used for sum, multiplication, or accumulation logic.

8. ANYMATCH() / ALLMATCH()

Used for conditional checks.

```
List<Integer> numbers = List.of(2,4,6,8);

boolean allEven =
    numbers.stream()
        .allMatch(n -> n % 2 == 0);

System.out.println(allEven);
```

These operations return boolean results.

9. MINI REAL EXAMPLE

```
List<String> names = List.of(
    "Rahul", "Amit", "Ankit", "Rohit"
);

List<String> result =
    names.stream()
        .filter(n -> n.startsWith("A"))
        .map(String::toUpperCase)
        .sorted()
        .toList();

System.out.println(result);
```

This pipeline filters, transforms, sorts, and collects data using a single readable flow.

WANT PART 2? 🙌

Comment the topic you want next:

- HashMap Internal Working Cheatsheet
- Java Multithreading Cheatsheet
- Java Collections Cheatsheet
- Spring Boot Annotations Cheatsheet

Your comments decide the next post 🚀

Save & Share with your React friends.